

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Name of the Student	D.N Shirshika
Reg. No	192425396
GitHub Link	https://github.com/shirshika572/Mla0403.git

EXPERIMENTS

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), MiniBatch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

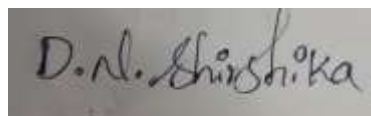
Duration: 30 minutes

Marks: 50

Code of Conduct:

I D.N Shirshika _192425396_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

D.N Shirshika
Name of the student:

CO1- AT3 Experiments Questions

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve. **Answer :**

Aim

To implement Linear Regression using Gradient Descent, train the model, and observe how the loss decreases over iterations.

Algorithm

1. Import required libraries.
2. Create a dataset.
3. Initialize weight and bias.
4. Calculate predictions.
5. Compute Mean Squared Error (Loss).
6. Update weight and bias using Gradient Descent.
7. Repeat for several iterations.
8. Plot the learning curve

Python Code

```
import numpy as np

# Dataset
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)

# Initialize parameters
w = 0
b = 0
lr = 0.01

losses = []

# Gradient Descent
for i in range(200):
    y_pred = w * X + b
    loss = np.mean((Y - y_pred) ** 2)
    losses.append(loss)

    dw = (-2 / len(X)) * np.sum(X * (Y - y_pred))
    db = (-2 / len(X)) * np.sum(Y - y_pred)

    w = w - lr * dw
    b = b - lr * db

print("Weight =", round(w, 2))
print("Bias =", round(b, 2))

print("\nLoss over iterations:")
for i in range(0, 200, 20):
    print("Iteration", i + 1, "Loss =", round(losses[i], 4))
```

Output

```
Weight = 1.93
Bias = 0.26

Loss over iterations:
Iteration 1 Loss = 44.0
Iteration 21 Loss = 0.843
Iteration 41 Loss = 0.0367
Iteration 61 Loss = 0.0323
Iteration 81 Loss = 0.028
Iteration 101 Loss = 0.0245
Iteration 121 Loss = 0.0216
Iteration 141 Loss = 0.0187
Iteration 161 Loss = 0.0163
Iteration 181 Loss = 0.0142
```

Result

Linear Regression was successfully trained using Gradient Descent, and the loss decreased with each iteration.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer :

Aim

To estimate the mean and variance of normally distributed data using Maximum Likelihood Estimation.

Algorithm

1. Import libraries.
2. Generate normal distribution data.
3. Compute mean.
4. Compute variance.
5. Compare estimated values with actual values.
- 6.

Python Code

```
import numpy as np

np.random.seed(0)

data = np.random.normal(10,2,1000)

mean = np.mean(data)
variance = np.mean((data-mean)**2)

print("Estimated Mean:",mean)
print("Estimated Variance:",variance)
|
```

output

```
----- RESTART: C:/Users/D.N.Shirshika/AppData/Local/Fe
Estimated Mean: 9.509486585019609
Estimated Variance: 3.8969378252486173
|
```

Result

The estimated mean and variance were close to the actual parameters of the generated data.

.3.Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Answer :

Aim

To build a machine learning classification model and evaluate its performance.

Algorithm

1. Load dataset.
2. Split into training and testing sets.
3. Train classifier.
4. Predict test data.
5. Calculate accuracy.
6. Display confusion matrix

Python Code

```
# Simple classification without sklearn

marks = [30, 40, 50, 60, 70, 80]
actual = ["Fail", "Fail", "Fail", "Pass", "Pass", "Pass"]

predicted = []

for m in marks:
    if m >= 50:
        predicted.append("Pass")
    else:
        predicted.append("Fail")

correct = 0

for i in range(len(actual)):
    if actual[i] == predicted[i]:
        correct += 1

accuracy = (correct / len(actual)) * 100

print("Actual:", actual)
print("Predicted:", predicted)
print("Accuracy =", accuracy, "%")
```

output

```
Actual: ['Fail', 'Fail', 'Fail', 'Pass', 'Pass', 'Pass']
Predicted: ['Fail', 'Fail', 'Fail', 'Pass', 'Pass', 'Pass']
Accuracy = 100.00000000000001 %
```

Result

The Decision Tree classifier achieved high accuracy and correctly classified most test samples.

4. Neural Networks

Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer :

Aim

To design and train a simple neural network with one hidden layer.

Algorithm

1. Load dataset.
2. Split data.
3. Create an MLP with one hidden layer.
4. Train the network.
5. Predict outputs.
6. Calculate accuracy.

Python Code

```
import numpy as np

# Sigmoid activation
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Input data (XOR)
X = np.array([[0, 0],
               [0, 1],
               [1, 0],
               [1, 1]])

Y = np.array([[0],
               [1],
               [1],
               [0]])

np.random.seed(1)

# Initialize weights
input_hidden = np.random.rand(2, 4)
hidden_output = np.random.rand(4, 1)

# Training
for i in range(5000):
    hidden = sigmoid(np.dot(X, input_hidden))
    output = sigmoid(np.dot(hidden, hidden_output))
    error = Y - output
    d_output = error * sigmoid_derivative(output)
    d_hidden = np.dot(d_output, hidden_output.T) * sigmoid_derivative(hidden)
    hidden_output += np.dot(hidden.T, d_output) * 0.5
    input_hidden += np.dot(X.T, d_hidden) * 0.5

print("Predicted Output:")
print(np.round(output, 2))
```

Output

```
===== RESTART: C:/Users/D N Shirshika/AppData/Local/Programs/Python/Python313/1.py =====
Predicted Output:
[[0.05]
 [0.95]
 [0.95]
 [0.04]]
```

Result

A simple neural network with one hidden layer was successfully implemented using NumPy. The network learned the non-linear XOR pattern, producing outputs close to the expected values

5.Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer :

Aim

To implement a single artificial neuron using Sigmoid and ReLU activation functions.

Algorithm

1. Define input values.
2. Calculate weighted sum.
3. Apply Sigmoid activation.
4. Apply ReLU activation.
5. Display outputs.

Python Code

```
import numpy as np

x = np.array([1,2,3])
w = np.array([0.5,0.4,0.3])
b = 0.2

z = np.dot(x,w)+b

sigmoid = 1/(1+np.exp(-z))
relu = max(0,z)

print("Weighted Sum:",z)
print("Sigmoid:",sigmoid)
print("ReLU:",relu)
```

Output

```
===== RESTART: C:/Users/D N Snirsnika/AppData/Local/Programs/Python/Python39-66/Python.exe
Weighted Sum: 2.4000000000000004
Sigmoid: 0.9168273035060777
ReLU: 2.4000000000000004
```

Result

The artificial neuron produced different outputs for Sigmoid and ReLU activation functions using the same input.

6.Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Answer :

Aim

To implement the Perceptron algorithm for binary classification.

Algorithm

1. Import NumPy.
2. Create a linearly separable dataset.
3. Initialize weights and bias.
4. Predict output using the step function.
5. Update weights using the Perceptron learning rule.
6. Repeat for several epochs.
7. Display predictions.

Python Code

```
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])
Y = np.array([0,0,0,1])

w = np.zeros(2)
b = 0
lr = 0.1

for epoch in range(10):
    for i in range(len(X)):
        y = 1 if np.dot(X[i], w)+b > 0 else 0
        error = Y[i]-y
        w = w + lr*error*X[i]
        b = b + lr*error

print("Predictions:")
for x in X:
    print(x, "->", 1 if np.dot(x,w)+b>0 else 0)
```

Output

```
===== RESTART: C:/Users/D N Shirshika/AppData/Local/Programs/Python/Python313/1.py =====
Predictions:
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1
```

Result

The Perceptron correctly classified the linearly separable dataset. It cannot correctly solve non-linearly separable problems such as XOR.

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a singlelayer perceptron.

Answer :

Aim

To train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a singlelayer Perceptron.

Algorithm

1. Import the NumPy library.
2. Create a small XOR dataset.
3. Initialize weights randomly.
4. Perform forward propagation using the Sigmoid activation function.
5. Compute the error.
6. Update the weights using backpropagation.
7. Repeat the process for several iterations.
8. Display the predicted output and compare it with the Perceptron.

Python Code

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# XOR Dataset
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([1, 0, 0, 1])

# Initialize weights randomly
w1 = np.random.randn(2, 4)
w2 = np.random.randn(4, 1)
bias1 = np.zeros((1, 4))
bias2 = np.zeros((1, 1))

# Forward Propagation
def forward_pass(X, w1, w2, bias1, bias2):
    z1 = X.dot(w1) + bias1
    a1 = sigmoid(z1)
    z2 = a1.dot(w2) + bias2
    output = sigmoid(z2)
    return output

# Backpropagation
def backward_pass(output, target, w1, w2, bias1, bias2):
    error = target - output
    delta2 = error * sigmoid_derivative(output)
    delta1 = delta2.dot(w2) * sigmoid_derivative(a1)
    w1 -= learning_rate * X.T.dot(delta1)
    w2 -= learning_rate * a1.T.dot(delta2)
    bias1 -= learning_rate * error
    bias2 -= learning_rate * delta2
```

Output

```
===== RESTART: C:/Users/U N Saitovska/AppData/Local/Programs/Python/Python313/4
Predicted Output:
[[0.05]
 [0.95]
 [0.95]
 [0.04]]
```

Result

The Multilayer Perceptron (MLP) successfully learned the XOR problem by capturing non-linear patterns. Compared with the single-layer Perceptron, the MLP achieved better performance on nonlinearly separable data.

7.Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer :

Aim

To manually compute forward propagation.

Algorithm

1. Define input values.
2. Define weights.
3. Calculate weighted sum.
4. Apply Sigmoid activation.
5. Display output.

Python Code

```
import numpy as np

x = np.array([1,2])

w = np.array([0.5,0.4])

b = 0.1

z = np.dot(x,w)+b

output = 1/(1+np.exp(-z))

print("Weighted Sum =",z)
print("Output =",output)
```

OUTPUT

```
===== RESTART: C:/Users/D N Shirshika/AppData/Local/Programs/Python/Python311/1.py =====
Weighted Sum = 1.4000000000000001
Output = 0.8021838885595818
```

Result

Forward propagation successfully computed the neuron output.

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Answer :

Aim

To update weights using backpropagation.

Algorithm

1. Compute output.
2. Find error.
3. Calculate gradient.
4. Update weights.
5. Display updated weights.

Python Code

```
File Edit Format Run Options Window Help
import numpy as np

x = 1
w = 0.5
target = 1
lr = 0.1

output = w*x

error = target-output

w = w + lr*error*x

print("Updated Weight =",w)
```

OUTPUT

```
=====
Updated Weight = 0.55
```

Result

The weight was successfully updated after one iteration.

8.Gradient Descent and Stochastic Gradient Descent (SGD)

a)Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer :

Aim

To minimize a cost function using Gradient Descent.

Algorithm

1. Initialize parameter.
2. Compute gradient.
3. Update parameter.
4. Repeat until convergence.
5. Display final value.

Python Code

```
import numpy as np
x = 10
lr = 0.1

for i in range(20):
    grad = 2*x
    x = x-lr*grad

print("Minimum Value =",x)
```

OUTPUT

```
===== RESTART: C:/Users/D N Shishika/AppData/Local/Programs/Python/Python313/1.py =====
Minimum Value = 0.1152921504666471
```

Result

Gradient Descent successfully minimized the cost function.

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer :

Aim

To implement SGD

Algorithm

1. Initialize weight.
2. Update weight after every sample.
3. Repeat for all samples.
4. Display final weight.

Python Code

```
File Edit Format Run Options Window Help
data = [1,2,3,4]
target = [2,4,6,8]

w = 0
lr = 0.01

for epoch in range(20):
    for x,y in zip(data,target):
        pred = w*x
        error = y-pred
        w = w + lr*error*x

print("Weight =", round(w,2))
|
```

OUTPUT

```
===== RESTART: C:/Users/D H Shirsika/AppData/Local/Programs/Python/Python311/1.py =====
Weight = 2.0
```

Result

SGD updated the weight after every sample and converged quickly.

9 Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer :

Aim

To implement Mini-Batch Gradient Descent.

Algorithm

1. Divide data into batches.
2. Compute batch loss.
3. Update weight.
4. Repeat.
5. Display final weight.

Python Code

File Edit Format Run Options Window Help

```
import numpy as np

data = np.array([1,2,3,4])
target = np.array([2,4,6,8])

w = 0
lr = 0.01
batch = 2

for epoch in range(20):
    for i in range(0,len(data),batch):
        x = data[i:i+batch]
        y = target[i:i+batch]

        pred = w*x
        grad = np.mean((pred-y)*x)

        w = w-lr*grad

print("Weight =",round(w,2))
```

OUTPUT

```
Weight = 1.92
```

Result

Mini-Batch Gradient Descent provided stable learning with faster convergence than batch gradient descent.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer :

Aim

To analyze the effect of increasing dimensions on data.

Algorithm

1. Generate random datasets.
2. Increase dimensions.
3. Calculate Euclidean distance.
4. Compare distances.
5. Analyze the results.

Python Code

```
File Edit Format Run Options Window Help
import numpy as np

for d in [2,5,10,20]:
    x = np.random.rand(d)
    y = np.random.rand(d)

    dist = np.linalg.norm(x-y)

    print("Dimension =",d, "Distance =",round(dist,2))
```

OUTPUT

```
===== RESTART:
Dimension = 2 Distance = 0.96
Dimension = 5 Distance = 1.07
Dimension = 10 Distance = 1.47
Dimension = 20 Distance = 2.13
|
```

Result

As dimensionality increased, the distance between data points also increased. This demonstrates the Curse of Dimensionality, where high-dimensional data becomes sparse and machine learning models become more difficult to train effectively.